Université de Yaoundé 1

Master 1 SIGL

INF461 : Principes et processus du logiciel

# Devoir n°1 : Patrons de construction

Patron Factory : Factory Method et Factory

**Étudiant :** KOUAWA BOUDA RYAN KEVIN

**Matricule :** 23U2557

**Dépôt GitHub :**

https://github.com/kev786/TD-INF461/tree/main/Patrons_De_construction/Devoir_1

Dossier Factory Method :
https://github.com/kev786/TD-INF461/tree/main/Patrons_De_construction/Devoir_1/factory-method

Dossier Factory :
https://github.com/kev786/TD-INF461/tree/main/Patrons_De_construction/Devoir_1/factory

Date de remise : 11/11/2026

# Introduction

Ce rapport présente le patron de construction **Factory** dans les deux implémentations vues en cours. Pour chacune, on donne l'exemple du cours (diagramme de classes et code Java), l'exemple modifié par l'ajout d'un **produit D** (diagramme et code), puis une analyse du modèle générique utilisé.

**1. Factory Method** : une seule fabrique dont la méthode de création reçoit un paramètre pour déterminer le type de l'objet à créer.
**2. Factory** : une fabrique abstraite dont les sous-classes (fabriques concrètes) créent chacune un type de produit.

Le code complet est disponible dans le dépôt GitHub indiqué sur la page de garde : un dossier par patron (`factory-method` et `factory`), chacun contenant l'exemple du cours (dossier `src/cours`) et l'exemple modifié (dossier `src/modifie`).

# 1. Patron Factory Method (méthode de création avec paramètre)

## 1.1 Exemple vu en cours

Le système gère trois types de produits. Le programme qui les manipule ne connaît le type à instancier qu'à l'exécution. La classe `FabriqueProduit` centralise la création : sa méthode `createProduit(String type)` choisit la classe à instancier et renvoie toujours un `Produit` (type parent).

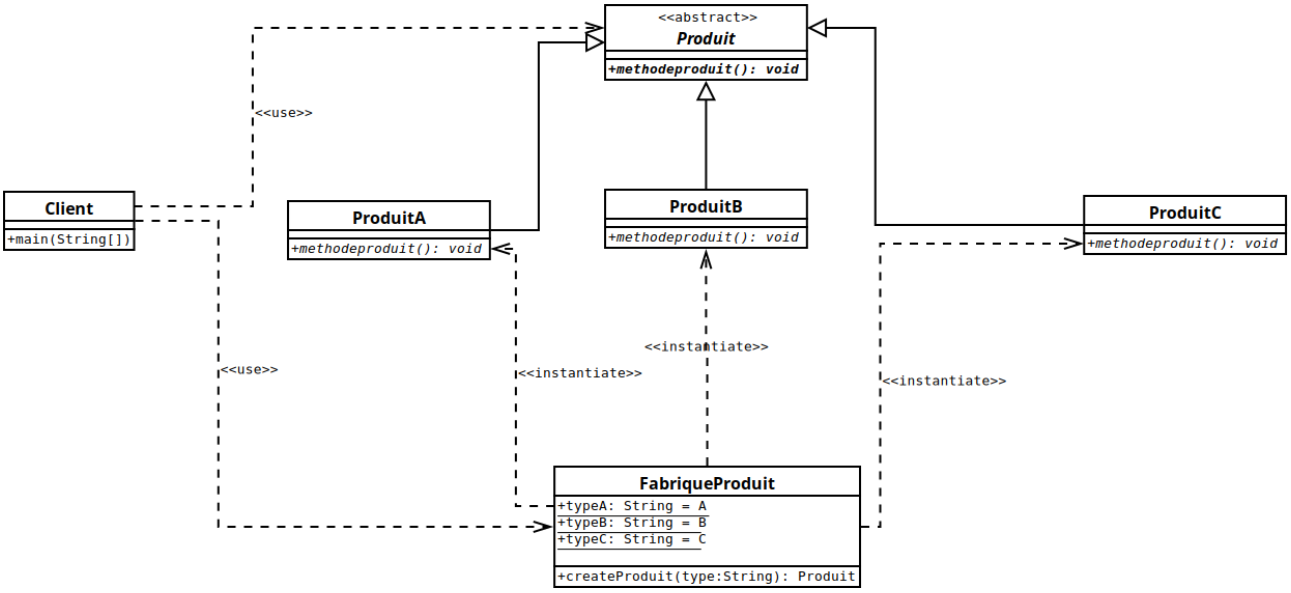



*Figure 1 : diagramme de classes, Factory Method (exemple du cours)*

***Implémentation :***

**`cours/Produit.java`**

```
package cours;

public abstract class Produit {
    public abstract void methodeproduit();
}
```

**`cours/ProduitA.java`**

```
package cours;

public class ProduitA extends Produit {

    public ProduitA() {
    }

    public void methodeproduit() {
        System.out.println("je suis un produit de type A");
        System.out.println("produit A.methodeproduit()");
    }

}
```

**cours/ProduitB.java**

```
package cours;

public class ProduitB extends Produit {

    public void methodeproduit() {
        System.out.println("je suis un produit de type B");
        System.out.println("produit B.methodeproduit()");
    }
}
```

**cours/ProduitC.java**

```
package cours;

public class ProduitC extends Produit {

    public void methodeproduit() {
        System.out.println("je suis un produit de type C");
        System.out.println("produit C.methodeproduit()");
    }
}
```

**cours/FabriqueProduit.java**

```
package cours;

public class FabriqueProduit {
    public static final String typeA = "A";
    public static final String typeB = "B";
    public static final String typeC = "C";

    public Produit createProduit(String type) {
        Produit produit = null;
        switch (type) {
            case typeA:
                produit = new ProduitA();
                break;
            case typeB:
                produit = new ProduitB();
                break;
            case typeC:
                produit = new ProduitC();
                break;
            default:
                throw new IllegalArgumentException("Type de produit non supporté");
        }
        return produit;
    }
}
```

**cours/Client.java**

```
package cours;

public class Client {

    public static void main(String[] args){

        System.out.println("bonjour le monde");

        FabriqueProduit fabriqueproduit = new FabriqueProduit();
        Produit produit = null;

        //affiche un produit de type A
        produit = fabriqueproduit.createProduit(FabriqueProduit.typeA);
        produit.methodeproduit();

        //affiche un produit de type B
        produit = fabriqueproduit.createProduit(FabriqueProduit.typeB);
        produit.methodeproduit();

        //affiche un produit de type C
        produit = fabriqueproduit.createProduit(FabriqueProduit.typeC);
        produit.methodeproduit();

        //produit inexistant
        String d= "D";
        //affiche un produit de type D
        produit = fabriqueproduit.createProduit(d);
        produit.methodeproduit();
    }
}
```

## 1.2 Exemple modifié : ajout du produit D

On ajoute un quatrième produit. Il faut : (1) créer la classe `ProduitD` qui hérite de `Produit` ; (2) ajouter un nouveau `case "D"` dans le `switch` de `FabriqueProduit`. Les classes `Produit`, `ProduitA`, `ProduitB` et `ProduitC` restent inchangées ; le `Client` demande simplement le type D.

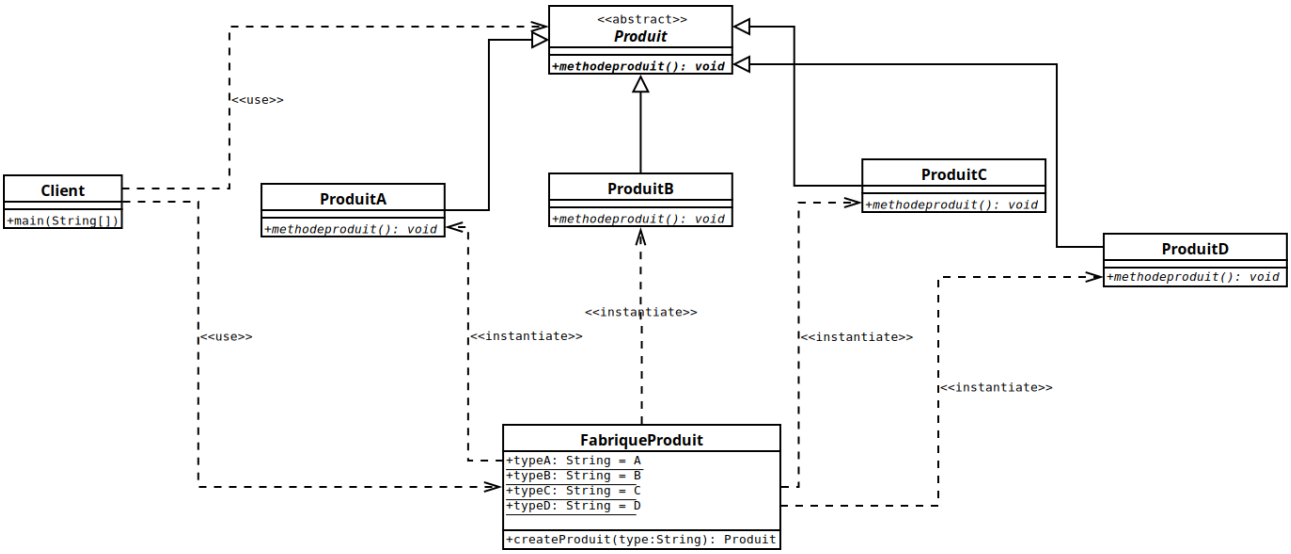



*Figure 2 : diagramme de classes, Factory Method (exemple modifié avec ProduitD)*

### *Implémentation (classes nouvelles ou modifiées) :*

**`modifie/ProduitD.java`**

```
package modifie;

public class ProduitD extends Produit {

    public ProduitD() {
    }

    public void methodeproduit() {
        System.out.println("je suis un produit de type D");
        System.out.println("produit D.methodeproduit()");
    }
}
```

**modifie/FabriqueProduit.java**

```
package modifie;

public class FabriqueProduit {
    public static final String typeA = "A";
    public static final String typeB = "B";
    public static final String typeC = "C";

    public Produit createProduit(String type) {
        Produit produit = null;
        switch (type) {
            case typeA:
                produit = new ProduitA();
                break;
            case typeB:
                produit = new ProduitB();
                break;
            case typeC:
                produit = new ProduitC();
                break;
            case "D":                       // <- ajout du produit D
                produit = new ProduitD();
                break;
            default:
                throw new IllegalArgumentException("Type de produit non supporté");
        }
        return produit;
    }
}
```

**modifie/Client.java**

```
package modifie;

public class Client {

    public static void main(String[] args){

        System.out.println("bonjour le monde");

        FabriqueProduit fabriqueproduit = new FabriqueProduit();
        Produit produit = null;

        //affiche un produit de type A
        produit = fabriqueproduit.createProduit(FabriqueProduit.typeA);
        produit.methodeproduit();

        //affiche un produit de type B
        produit = fabriqueproduit.createProduit(FabriqueProduit.typeB);
        produit.methodeproduit();

        //affiche un produit de type C
        produit = fabriqueproduit.createProduit(FabriqueProduit.typeC);
        produit.methodeproduit();

        //affiche un produit de type D
        produit = fabriqueproduit.createProduit("D"); // AJOUT DU PRODUIT D
        produit.methodeproduit();

        //produit inexistant
        String e= "e";
        //affiche un produit de type E
        produit = fabriqueproduit.createProduit(e);
        produit.methodeproduit();
    }
}
```

## Analyse : choix du modèle générique (Factory Method)

Le modèle générique utilisé est celui du cours : **Client**, **Produit**, **ProduitConcret** et **Fabrique** avec une méthode de fabrication `methodeFabrique(): Produit`. Il convient ici parce que :

• le type de produit n'est connu qu'à l'exécution, ce que la fabrique permet de gérer à partir d'un paramètre (`type`) ;
• tous les `new` sont regroupés à un seul endroit, la fabrique, ce qui évite au client de dépendre des classes concrètes `ProduitA/B/C/D` (couplage faible : il ne manipule que le type abstrait `Produit`) ;
• le polymorphisme permet de spécialiser le traitement : `methodeproduit()` se comporte différemment selon le produit.
• c'est une solution simple (une seule fabrique) adaptée à un nombre limité de types.

**Limite :** l'ajout du produit D oblige à modifier la fabrique (nouveau `case`). Le client n'a pas à changer pour créer les produits, mais la classe `FabriqueProduit` n'est pas fermée à la modification. Cette limite est levée par le patron Factory (partie suivante).

# 2. Patron Factory (fabrique abstraite et fabriques concrètes)

## 2.1 Exemple vu en cours

Ici, la fabrique est une classe abstraite `ProduitFactory` qui contient la méthode `getProduit()` (écrite une seule fois) et la méthode abstraite `createProduit()`. Chaque sous-classe (`ProduitFactoryA/B/C`) redéfinit `createProduit()` pour créer son propre produit. Le choix du produit se fait en choisissant la fabrique à instancier.

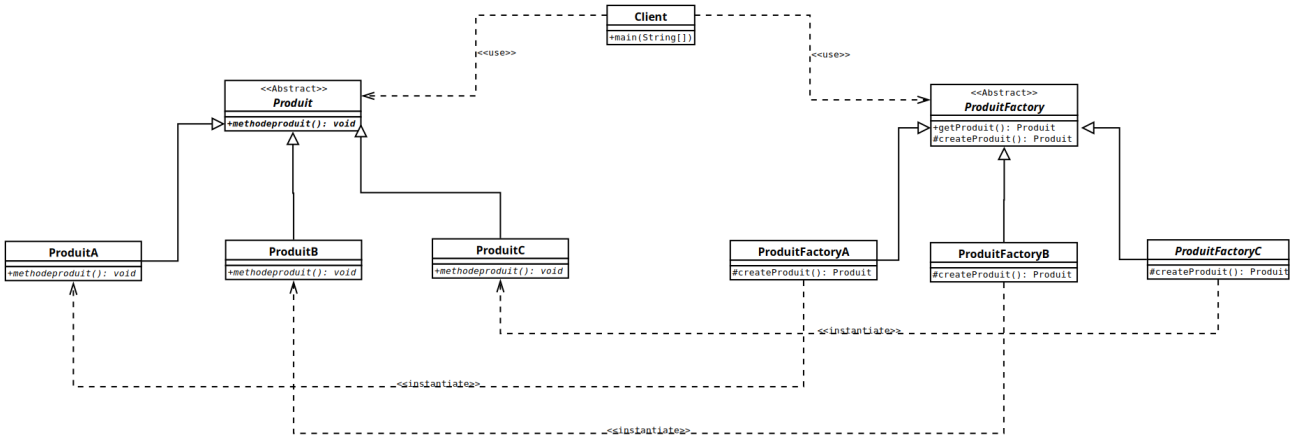



*Figure 3 : diagramme de classes, Factory (exemple du cours)*

### *Implémentation :*

**cours/Produit.java**

```
package cours;

public abstract class Produit {
    public abstract void methodeproduit();
}
```

**cours/ProduitA.java**

```
package cours;

public class ProduitA extends Produit {


    public void methodeproduit() {
        // Implementation for ProduitA
        System.out.println("je suis un produit de type A");
        System.out.println("produitA.methodeproduit()");
    }

}
```

**cours/ProduitB.java**

```
package cours;

public class ProduitB extends Produit {

    public void methodeproduit() {
        // Implementation for ProduitB
        System.out.println("je suis un produit de type B");
        System.out.println("produitB.methodeproduit()");
    }

}
```

**cours/ProduitC.java**

```
package cours;

public class ProduitC extends Produit {

    public void methodeproduit() {
        // Implementation for ProduitC
        System.out.println("je suis un produit de type C");
        System.out.println("produitC.methodeproduit()");
    }

}
```

**cours/ProduitFactory.java**

```
package cours;

public abstract class ProduitFactory {
    public Produit getProduit(){
        return createProduit();
    }

    protected abstract Produit createProduit();
}
```

**cours/ProduitFactoryA.java**

```
package cours;

public class ProduitFactoryA extends ProduitFactory {

    protected Produit createProduit() {
        Produit produit = null;
        produit = new ProduitA();
        return produit;
    }

}
```

**cours/ProduitFactoryB.java**

```
package cours;

public class ProduitFactoryB extends ProduitFactory {

    protected Produit createProduit() {
        Produit produit = null;
        produit = new ProduitB();
        return produit;
    }

}
```

**cours/ProduitFactoryC.java**

```
package cours;

public class ProduitFactoryC extends ProduitFactory {

    protected Produit createProduit() {
        Produit produit = null;
        produit = new ProduitC();
        return produit;
    }
}
```

**cours/Client.java**

```
package cours;

public class Client {
    public static void main(String[] args) {
        System.out.println(".......Methode 2 : Abstract Factory.......");
        ProduitFactory produitfactoryA = new ProduitFactoryA();
        ProduitFactory produitfactoryB = new ProduitFactoryB();
        ProduitFactory produitfactoryC = new ProduitFactoryC();
        Produit produit = null;

        //affiche un produit de type A
        produit = produitfactoryA.getProduit();
        produit.methodeproduit();

        //affiche un produit de type B
        produit = produitfactoryB.getProduit();
        produit.methodeproduit();

        //affiche un produit de type C
        produit = produitfactoryC.getProduit();
        produit.methodeproduit();


    }
}
```

## 2.2 Exemple modifié : ajout du produit D

Pour ajouter le produit D, on **crée deux nouvelles classes** : `ProduitD` (qui hérite de `Produit`) et `ProduitFactoryD` (qui hérite de `ProduitFactory` et crée un `ProduitD`). Aucune classe existante de la hiérarchie n'est modifiée ; seul le `Client` est complété pour utiliser la nouvelle fabrique.

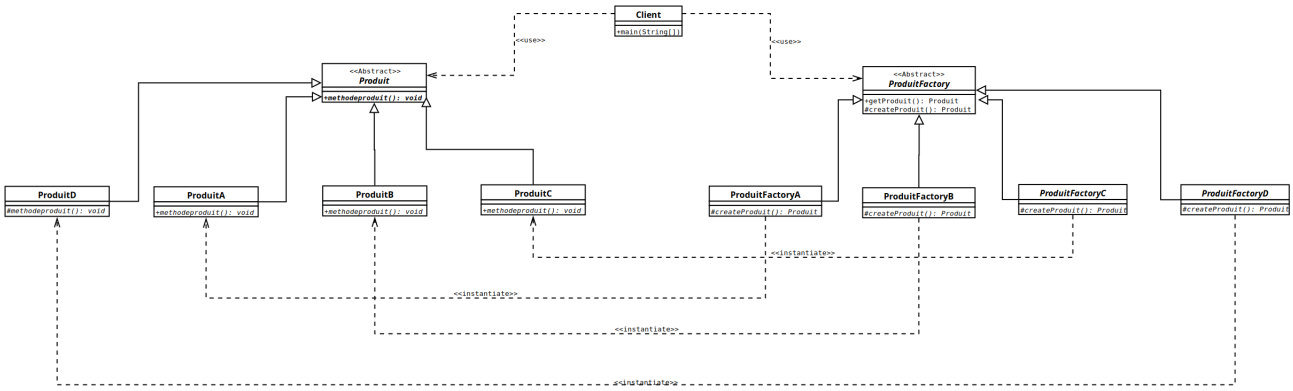



*Figure 4 : diagramme de classes, Factory (exemple modifié avec ProduitD et ProduitFactoryD)*

### *Implémentation (classes nouvelles ou modifiées) :*

**`modifie/ProduitD.java`**

```
package modifie;

//AJOUT DE LA CLASSE PRODUITD
public class ProduitD extends Produit {

    public void methodeproduit() {
        System.out.println("je suis un produit de type D");
        System.out.println("produitD.methodeproduit()");
    }

}
```

**`modifie/ProduitFactoryD.java`**

```
package modifie;

//AJOUT DE LA CLASSE PRODUITFACTORYD
public class ProduitFactoryD extends ProduitFactory {
    protected Produit createProduit() {
        Produit produit = null;
        produit = new ProduitD();
        return produit;
    }

}
```

**modifie/Client.java**

```
package modifie;

public class Client {
    public static void main(String[] args) {
        System.out.println("........Methode 2 : Abstract Factory........");
        ProduitFactory produitfactoryA = new ProduitFactoryA();
        ProduitFactory produitfactoryB = new ProduitFactoryB();
        ProduitFactory produitfactoryC = new ProduitFactoryC();
        ProduitFactory produitfactoryD = new ProduitFactoryD(); // AJOUT DE LA FABRIQUE D
        Produit produit = null;

        //affiche un produit de type A
        produit = produitfactoryA.getProduit();
        produit.methodeproduit();

        //affiche un produit de type B
        produit = produitfactoryB.getProduit();
        produit.methodeproduit();

        //affiche un produit de type C
        produit = produitfactoryC.getProduit();
        produit.methodeproduit();

        //affiche un produit de type D
        produit = produitfactoryD.getProduit(); // AJOUT DU PRODUIT D
        produit.methodeproduit();


    }
}
```

## Analyse : choix du modèle générique (Factory)

Le modèle générique utilisé est celui du cours : **FabriqueAbstraite** (signature de la méthode de fabrique et méthodes qui l'invoquent), **FabriqueConcrete**, **Produit**, **ProduitConcret** et **Client**. Il convient ici parce que :

• on veut pouvoir **ajouter des produits sans modifier le code existant** (principe ouvert/fermé) : pour le produit D il suffit d'ajouter `ProduitD` et `ProduitFactoryD` ;
• il n'y a plus d'instruction conditionnelle (`switch`) à maintenir ;
• le client dépend uniquement des types abstraits `Produit` et `ProduitFactory`, jamais des produits concrets ;
• la méthode `getProduit()` est écrite une seule fois dans la classe abstraite et le polymorphisme sélectionne la bonne `createProduit()`.

**Limite :** le nombre de classes augmente (une fabrique concrète par produit).

## Comparaison

Factory Method (avec paramètre) est plus simple et suffit quand les types changent peu ; Factory (fabrique abstraite et sous-classes) est préférable quand les types de produits sont susceptibles d'évoluer, car elle permet l'extension sans modification du code existant.

## Conclusion

Dans les deux cas, le client ne crée jamais les produits avec l'opérateur `new` : il passe par une fabrique et manipule seulement le type abstrait `Produit`. Cela réduit le couplage et rend le code plus facile à faire évoluer.